

Additional Programming techniques

Study Guide

Prof. Saurav Bharadwaj
Assistant Professor
Dept of ECE, PIET
Parul University

1. Counter	1
2. Time delay	2
3. Stack and Subroutines	3
4. Restart, Call and Return Instruction	5
5. Code conversion	7

4.1 Counter

- Counter refers to a program counter (PC) or a software-based counter implemented using instructions. Counters are used to keep track of the number of events or repetitions in a program.

Program Counter (PC) – Hardware Counter

- Program Counter (PC) is a 16-bit register in the 8085 microprocessor that stores the address of the next instruction to be executed.
- It acts as a counter since it increments automatically after each instruction fetch.
- It helps in sequential execution of instructions.
- During jumps or branches, it is loaded with a new address.

Example:

Memory Address	Instruction
2000H	MVI A, 32H
2002H	MVI B, 12H
2004H	ADD B
2005H	HLT

Software Counter – Using Register as a Counter

Example: Counting Down from 05H to 00H

MVI C, 05H	Load 5 into register C (acts as counter)
LOOP: DCR C	Decrease C by 1
JNZ LOOP	Jump to LOOP if C ≠ 0
HLT	Stop execution

Example: Up Counter

MVI C, 00H	Initialize counter to 0
MVI B, 05H	Set upper limit to 5
LOOP: INR C	Increment counter
CMP B	Compare C with B
JNZ LOOP	Repeat if not equal
HLT	Stop execution

Use Cases of Counters in 8085

- Looping through data in memory (e.g., summing 10 numbers).
- Delays (by counting iterations).
- Counting external events (with external hardware + interrupt).

4.2 Time Delay

Time delay in the context of the 8085 microprocessor refers to a deliberate pause introduced in program execution, typically for timing control, such as creating a delay between operations (e.g., blinking an LED or waiting for hardware to be ready).

To create a delay, the microprocessor runs a loop that performs a number of instructions, each taking a known number of T-states (clock cycles). The total delay depends on:

- The clock frequency of the microprocessor.
- The number of T-states consumed by each instruction.
- The number of times the loop executes.

For example: If the 8085 is running at 3 MHz, one T-state = $1 / 3 \text{ MHz} = 0.333 \text{ microseconds}$.

Delay Calculation

Example 1: Simple Loop

MVI C, FFH	7 T-states
LOOP: DCR C	4 T-states
JNZ LOOP	10 (if jump taken) / 7 (if not taken)

Assume the clock frequency = 3 MHz.

Breakdown:

MVI C, FFH	7 T-states
Loop runs 255 times (from FFH down to 00H)	

Inside the loop:

DCR C – 4 T-states

JNZ LOOP – 10 T-states if jump is taken, 7 T-states if not taken

So for 254 iterations (jump taken):

$$254 \times (4 + 10) = 254 \times 14 = 3556 \text{ T-states}$$

Final iteration (jump not taken):

4 (DCR) + 7 (JNZ not taken) = 11 T-states

Total delay = 7 (initial MVI) + 3556 + 11 = 3574 T-states

Convert to time:

Delay = $3574 \times 0.333 \mu\text{s} \approx 1190 \mu\text{s} \approx 1.19 \text{ ms}$

Example 2: Nested loop for larger delay

MVI B, 0AH	Outer loop counter (10)
LOOP1: MVI C, FFH	Inner loop counter (255)
LOOP2: DCR C	
JNZ LOOP2	
DCR B	
JNZ LOOP1	

Breakdown:

MVI B, 0AH – 7 T-states

MVI C, FFH – 7 T-states (inside outer loop)

Inner loop:

$254 \times (4 + 10) + 4 + 7 = 3574$ T-states (same as above)

Outer loop runs 10 times:

$10 \times (7 + 3574) = 10 \times 3581 = 35810$ T-states

Add DCR B + final JNZ B (when false): $4 + 7 = 11$

Total = $7 + 35810 + 11 = 35828$ T-states

Convert to time:

Delay = $35828 \times 0.333 \mu\text{s} \approx 11927 \mu\text{s} \approx 11.93 \text{ ms}$

4.3 Stack and Subroutines

In the 8085 microprocessor, stack and subroutines are fundamental concepts used for managing temporary data storage and implementing modular code. The stack is a portion of RAM used for temporary storage. It works on LIFO (Last In, First Out) principle. The Stack Pointer (SP) holds the address of the top of the stack.

Instructions:

Instruction	Description
PUSH rp	Push contents of register pair onto stack
POP rp	Pop contents from stack into register pair
XTHL	Exchange top of stack with HL
SPHL	Copy contents of HL into SP
PCHL	Jump to address contained in HL (can be used to simulate subroutine calls)

Example:

```
LXI SP, 5000H ; Initialize Stack Pointer
LXI H, 1234H ; Load HL with 1234H
PUSH H ; Push HL onto stack
POP D ; Pop into DE register pair
```

Subroutines are reusable blocks of code. When a subroutine is called, the current program counter (PC) is saved on the stack, and control jumps to the subroutine. After execution, control returns using RET.

Instruction	Description
CALL addr	Call subroutine at specified address
RET	Return from subroutine
RST n	Call a subroutine at fixed address ($n \times 8$)

Example:

```
LXI SP, 5000H ; Initialize stack
CALL SUB_ROUTINE ; Call subroutine
```

Main code continues

SUB_ROUTINE:

```
MVI A, 0AH ; Load 0AH into Accumulator
RET ; Return to caller
```

Comparison between stack and sub-routine

Aspect	Stack	Subroutine
Definition	A special area in memory used for temporary data storage (LIFO).	A block of code that performs a specific task and can be called from different places in the program.
Purpose	To save/restore data temporarily, especially during subroutine calls, interrupts, etc.	To make code modular, reusable, and structured.

Aspect	Stack	Subroutine
Managed by	Stack Pointer (SP) register	Program Counter (PC) and CALL/RET instructions
Key Instructions	PUSH, POP, XTHL, SPHL	CALL, RET, RST
Memory Use	Uses RAM locations to store register contents or return addresses	Code segment located elsewhere in memory
Data Handled	Typically register contents, return addresses	Instructions (code) to be executed
Entry and Exit	No automatic entry/exit—done using PUSH and POP	Entered with CALL, exited with RET
Relation	Supports subroutines by storing return addresses and register values	Uses stack to return to the main program
Example Use Case	Saving HL before modifying it temporarily	Performing a calculation used in many parts of the program
Storage Type	Temporary, volatile	Part of the permanent program structure

- A subroutine depends on the stack to save the return address.
- The stack can be used independently, but it is often utilized in subroutines to save/restore register contents.

4.4 Restart, Call and Return

In the 8085 microprocessor, Restart, Call, and Return instructions are used for subroutine handling and interrupt processing.

Restart Instructions (RST)

Mnemonic	Opcode	Address Vector
RST 0	C7	0000H
RST 1	CF	0008H
RST 2	D7	0010H
RST 3	DF	0018H
RST 4	E7	0020H
RST 5	EF	0028H
RST 6	F7	0030H
RST 7	FF	0038H

Call Instructions

Mnemonic	Description
CALL addr	Unconditional call to a subroutine at addr
CC addr	Call if Carry = 1

Mnemonic	Description
CNC addr	Call if Carry = 0
CZ addr	Call if Zero = 1
CNZ addr	Call if Zero = 0
CP addr	Call if Sign = 0 (Positive)
CM addr	Call if Sign = 1 (Minus)
CPE addr	Call if Parity = 1 (Even)
CPO addr	Call if Parity = 0 (Odd)

Return Instructions

Mnemonic	Description
RET	Unconditional return from subroutine
RC	Return if Carry = 1
RNC	Return if Carry = 0
RZ	Return if Zero = 1
RNZ	Return if Zero = 0
RP	Return if Sign = 0 (Positive)
RM	Return if Sign = 1 (Minus)
RPE	Return if Parity = 1 (Even)
RPO	Return if Parity = 0 (Odd)

In the 8085 microprocessor, the instructions related to Restart (RST), Call, and Return are crucial for implementing subroutines and handling interrupts efficiently. These instructions allow a programmer to transfer control of execution to a different part of the program, execute a specific task, and then return to the original sequence.

Restart (RST) instructions are one-byte instructions primarily used for handling hardware and software interrupts. There are eight such instructions, from RST 0 to RST 7, each corresponding to a fixed memory location that is a multiple of 8 (starting from address 0000H to 0038H). When an RST instruction is executed, the current program counter (PC) is saved onto the stack, and execution jumps to the corresponding address. This mechanism allows quick response to interrupts without needing a full address, making RST instructions compact and efficient.

Call instructions are used to invoke subroutines, which are separate blocks of code intended to perform a specific task. The basic CALL instruction is unconditional and transfers control to a specified memory address, saving the return address (i.e., the address of the instruction following the CALL) on the stack. Conditional call instructions like CC (Call if Carry), CNC (Call if No Carry), CZ (Call if Zero), and others provide a way to conditionally execute subroutines based on the status of specific flags in the flag register. These are all three-byte instructions — one byte for the opcode and two for the 16-bit address of the subroutine.

Return instructions are used to bring back control from the subroutine to the calling program. The basic RET instruction pops the return address from the stack and jumps to it, resuming execution from where the call was made. Like the Call instructions, Return instructions also have conditional variants such as RC

(Return if Carry), RZ (Return if Zero), and others. These allow conditional exits from subroutines based on the processor's flag status and are typically one-byte instructions.

4.5 Code Conversions

1. Binary to BCD Conversion

This converts an 8-bit binary number to its BCD representation.

Binary number in register C. Convert it to BCD and store hundreds in D, tens in E, units in A.

```
MVI C, 0xFA    ; Load binary number FAH (250 decimal)
MVI D, 00      ; Clear D (hundreds)
MVI E, 00      ; Clear E (tens)
```

```
LOOP1: CPI 64   ; Compare with 100 decimal
      JC CHECK_TENS
      SUI 64    ; Subtract 100
      INR D     ; Increment hundreds
      JMP LOOP1
```

```
CHECK_TENS:
      CPI 0AH   ; Compare with 10
      JC END
      SUI 0AH   ; Subtract 10
      INR E     ; Increment tens
      JMP CHECK_TENS
```

```
END:   ; A now has units, E has tens, D has hundreds
      HLT
```

2. BCD to Binary Conversion

To convert a BCD number (hundreds in D, tens in E, units in A) into binary.

Example:

Assume D = 0x02, E = 0x05, A = 0x04 (i.e., BCD = 254 decimal)

```
MVI D, 02      ; Hundreds
MVI E, 05      ; Tens
MVI A, 04      ; Units
```

```
MOV B, D       ; Copy hundreds to B
MVI C, 64      ; Load 100 decimal
CALL MUL       ; Multiply B × 100 → result in H
```

```
MOV B, E       ; Copy tens to B
MVI C, 0AH     ; Load 10 decimal
CALL MUL       ; Multiply B × 10 → add to H
```

ADD H ; Add units to the result in H
HLT

; Subroutine MUL: Multiply B × C, result in H

MUL:

MVI H, 00

MVI L, 00

LOOPM:

ADD H

DCR B

JNZ LOOPM

RET

3. ASCII to Binary Conversion

To convert an ASCII digit to binary (e.g., ASCII '5' = 0x35 → Binary 5).

MVI A, 35H ; Load ASCII of '5'

SUI 30H ; Subtract 30H to get binary

HLT

4. Binary to ASCII Conversion

To convert a binary digit to its ASCII equivalent (e.g., Binary 5 → ASCII '5' = 0x35).

MVI A, 05H ; Load binary number

ADI 30H ; Add 30H to get ASCII

HLT

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | **GRADE**



<https://paruluniversity.ac.in/>